

AI WEATHERWISE

Phase 4: Project Planning

Epics, sprints, timeline, resources and risks

Project	AI WeatherWise – Deliver intelligent weather forecasts and insights
Phase	4 of 8 – Project Planning
Technology	Node.js, Express.js, MongoDB (Mongoose), JWT, Google Gemini AI, OpenWeatherMap
Version / Date	1.0 28 September 2026

1. Objectives of the Plan

This plan describes how the AI WeatherWise backend is organized, scheduled and delivered. It breaks the work into epics and sprints, assigns responsibilities, identifies risks and defines when each deliverable is complete.

2. Scope and Deliverables

#	Deliverable	Description
D1	Backend source code	Node.js / Express / MongoDB API in a public GitHub repository.
D2	Phase-wise documents	Eight documents – from brainstorming to demonstration.
D3	Postman collection	postman_collection.json with all tested requests.
D4	API testing video	Recording of the Postman tests.
D5	Demo video	Backend demonstration video.

3. Epics and Work Breakdown

Epic	Title	Main tasks	Output
1	Project Architecture	Define layers, MVC pattern, components, ER model and user flow.	Architecture and ER diagrams.
2	Project Setup & Configuration	Create folder, run <code>npm init</code> , install dependencies, create <code>.env</code> , <code>.gitignore</code> and folder structure.	Ready project skeleton.
3	Backend Development	Implement entry point, app, routes, controllers, middleware, services.	Working REST API.
4	Database Configuration	Connect MongoDB with Mongoose; create User and Location schemas.	Persistent data layer.
5	Project Execution & Testing	Run the server; test register, login, weather and AI APIs in Postman.	Test evidence and videos.

4. Sprint Plan

Sprint	Focus	Tasks	Story points
Sprint 1 (Week 1–2)	Foundation	Ideation, requirements, architecture and data design; project setup, dependencies and <code>.env</code> .	13
Sprint 2 (Week 3–4)	Core backend	MongoDB connection, User and Location models, authentication (register/login, JWT), location CRUD.	21
Sprint 3 (Week 4–5)	Weather & AI	Weather service with fallback mode; Gemini AI service; recommendation endpoint; error handling and sanitization.	21
Sprint 4 (Week 6)	Test & deliver	Postman testing, bug fixing, documentation, GitHub upload, demo recording.	13

4.1 Timeline (Gantt view)

Activity	W1	W2	W3	W4	W5	W6
Brainstorming & requirements						
Design (architecture, ER, API)						
Setup & configuration						
Development (auth, location, weather, AI)						
Testing (Postman)						
Documentation						
Demo & submission						

5. Roles and Responsibilities

Role	Responsibility
Project owner / developer	Overall design, backend implementation, GitHub repository maintenance.
Tester	Prepares Postman requests, executes test cases, records defects and videos.
Documentation lead	Prepares the eight phase documents, README and demo script.
Reviewer / mentor	Reviews each phase deliverable before submission.

In a single-person project, one member may take all roles.

6. Tools and Technologies

Purpose	Tool
Runtime / framework	Node.js (v16+), Express.js
Database	MongoDB with Mongoose ODM

Purpose	Tool
Security	bcryptjs, jsonwebtoken, cors, dotenv
AI	Google Gemini SDK
Development	Visual Studio Code, Nodemon
Testing	Postman / Thunder Client
Version control	Git and a public GitHub repository

7. Milestones

Milestone	Completion criteria	Target
M1 – Requirements approved	Phase 1 and 2 documents reviewed	End of Week 2
M2 – Design frozen	Architecture, ER diagram and API list agreed	End of Week 3
M3 – Core API working	Register, login and location CRUD working	End of Week 4
M4 – AI integration complete	Weather and AI recommendation endpoints working	End of Week 5
M5 – Delivery	Tests passed, documents and videos uploaded to GitHub	End of Week 6

8. Risk Management

Risk	Likelihood	Impact	Mitigation
Missing or invalid weather / Gemini API key	Medium	High	Fallback (mock) mode keeps the API functional.
MongoDB not running or wrong URI	Medium	High	Fail-fast start-up message; default local URI; checklist before running.
Security flaws (weak secrets, injection)	Low	High	Strong JWT secret in <code>.env</code> , sanitization, bcrypt hashing.
Third-party API rate limits or downtime	Medium	Medium	Handle errors centrally; use fallback data.
Schedule slip	Medium	Medium	Prioritize "Must" requirements; weekly review.

9. Configuration and Version Control Plan

- All project files and documents are uploaded to a **public GitHub repository**.
- The project is maintained and submitted **phase-wise**, one folder or document per phase.
- `.env` and `node_modules` are excluded using `.gitignore`.
- Commits are small and descriptive, for example "Add Location model with unique index".

10. Definition of Done

- Code runs with `npm start` and connects to MongoDB.

- The feature is tested in Postman with the expected status code and response body.
- The feature is documented and pushed to GitHub.

The next phase, **Project Development**, executes this plan.